arXiv:2401.00448v3 [cs.LG] 14 Apr 2025

Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws

Nikhil Sardana [1] **Jacob Portes** [1] **Sasha Doubov** [1] **Jonathan Frankle** [1]

Abstract

Large language model (LLM) scaling laws are empirical formulas that estimate changes in model quality as a result of increasing parameter count and training data. However, these formulas, including the popular Deepmind Chinchilla scaling laws, neglect to include the cost of inference. We modify the Chinchilla scaling laws to calculate the optimal LLM parameter count and pre-training data size to train and deploy a model of a given quality and inference demand. We conduct our analysis both in terms of a compute budget and real-world costs and find that LLM researchers expecting reasonably large inference demand (~1B requests) should train models smaller and longer than Chinchilla-optimal. Furthermore, we train 47 models of varying sizes and parameter counts to validate our formula and find that model quality continues to improve as we scale tokens per parameter to extreme ranges (up to 10,000). Finally, we ablate the procedure used to fit the Chinchilla scaling law coefficients and find that developing scaling laws only from data collected at typical token/parameter ratios overestimates the impact of additional tokens at these extreme ranges.

1. Introduction

Large language models (LLMs) have substantial training and inference compute and energy costs (Knight, 2023; Pope et al., 2022). Training computation costs are primarily determined by the size of the model and the amount of data seen during training (Hoffmann et al., 2022). For state-of-the-art models with tens of billions of parameters trained on trillions of tokens, training costs can easily exceed millions of dollars. Similarly, inference costs depend on the size of the model and the volume of user queries over the lifetime of the model. This volume can be significant; demand for popular models can exceed billions of tokens per day (OpenAI & Pilipiszyn, 2021; Shazeer & Freitas, 2022).

Accounting for both *training and inference*, how does one minimize the cost required to produce and serve a high quality model?

Recent studies have proposed scaling laws, empirical formulas that estimate how changes in model and training data size impact model quality (Kaplan et al., 2020; Hoffmann et al., 2022). Hoffmann et al. (2022) is perhaps the most influential of these works, finding that to scale language models most efficiently, parameters and tokens should grow approximately linearly. The authors applied this scaling law to train a 70B parameter model (dubbed *Chinchilla*) that outperformed much larger and more expensive models such as GPT-3. As a result, many subsequent LLMs have been trained following the Chinchilla scaling laws (Dey et al., 2023; Muennighoff et al., 2023).

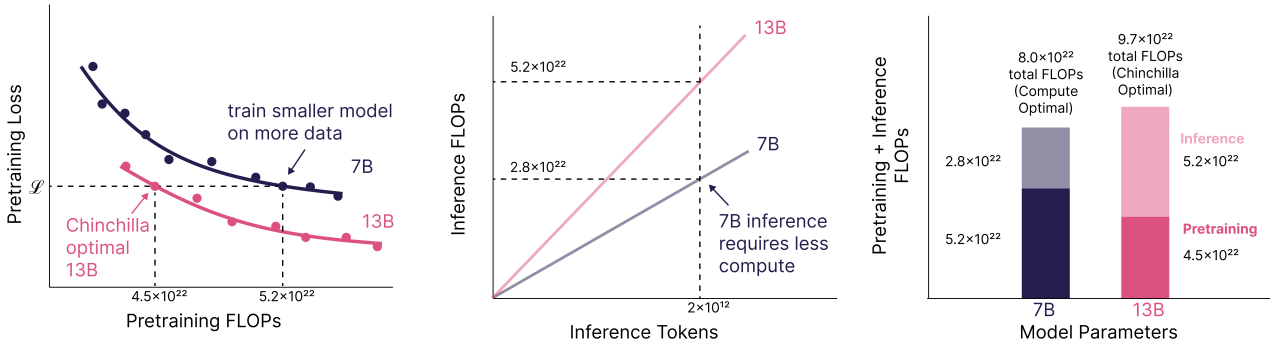
However, the Chinchilla scaling laws only account for the computational costs of training. By contrast, the Llama 2 family of models were trained on 2 trillion tokens and the Llama 3 family of models were trained on 15 trillion tokens, which is far more data than the Chinchilla scaling laws would deem "optimal" (Touvron et al., 2023a;b; AI@Meta, 2024). Since inference costs are lower for smaller models, the extra training compute required to train a Llama-style model over a Chinchilla-style model of equivalent quality pays off after enough inference requests.

Prior work has discussed the training-inference compute trade-off (Touvron et al., 2023a;b; Tow et al., 2023; De Vries, 2023; Villalobos & Atkinson, 2023). Touvron et al. (2023a) cites the lower inference cost of smaller models as inspiration for the LLaMA series. De Vries (2023) calculates the compute overhead of training longer than Chinchilla, but does not discuss quantify compute savings from inference. Villalobos & Atkinson (2023) discuss this trade-off in more detail, but show the shift in scaling laws for only a single particular number of inferences.

Other related work includes Muennighoff et al. (2023), which adapts the Chinchilla scaling laws for the data-constrained regime, where we have to repeat training tokens.

[1]Databricks MosaicML, United States of America. Correspondence to: Nikhil Sardana <nikhil@mosaicml.com>.

Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024. Copyright 2024 by the author(s).



(a) (b) (c)

Figure 1: Schematic of compute savings achieved via our approach. An LLM developer seeking to train a 13B model who expects 2 trillion tokens of inference demand during the model's lifetime can reduce their total compute by 1.7×10^{22} FLOPs (17%) by instead training a 7B model on more data (a). The extra compute required to train the 7B model beyond its Chinchilla-optimal point to match the 13B's quality is made up for during inference (b), (c). Our method quantifies this training-inference trade-off, producing models that are optimal over their total lifetime.

Our problem setting is the opposite: We assume we are *data rich* but *compute constrained*, and seek to minimize computation costs assuming we have enough data to train high-quality models.

In this paper, we modify the Chinchilla scaling laws to account for inference costs by calculating the optimal parameter and training token counts—both in terms of compute (Sec. 2) and dollar costs (Sec. 6)—to train and deploy a model of any given quality and inference demand. Our principled derivation estimates that LLM practitioners expecting significant demand (~10^9 inference requests) should train models substantially smaller and longer than Chinchilla-optimal. Figure 1 illustrates the benefits of our *compute-optimal* method for a realistic scenario.

In inference-heavy regimes, our modification predicts that it is optimal to train a model far smaller and longer than Chinchilla predicts, up to thousands of tokens per parameter. Does this hold true in practice? Do transformer models see continued improvements at such extreme cases? Or is there a point beyond which models saturate, and additional tokens provide no further improvement, as De Vries (2023) suggests? To uncover the behavior of transformer models in these cases, we train 47 models ranging from 150M to 6B parameters, on various data budgets from 10 to 10,000 tokens per parameter. We find that model quality continues to improve as we scale token ratios. We do not find evidence of a "saturation point," beyond which models do not improve, even with additional data.

Lastly, we ablate the parametric curve fitting procedure from the Chinchilla scaling laws. The Chinchilla scaling laws use empirical data from over 400 training runs to determine coefficients that estimate precisely how additional model parameters and training data impact loss. Hoffmann et al. (2022) collected data only from standard token ratio training runs (≤~100 tokens/parameters). Our ablation indicates that when fitting Chinchilla coefficients using only typical token ratio runs, this formula *overestimates* the impact of additional training data as we move to the extreme ratio regime.

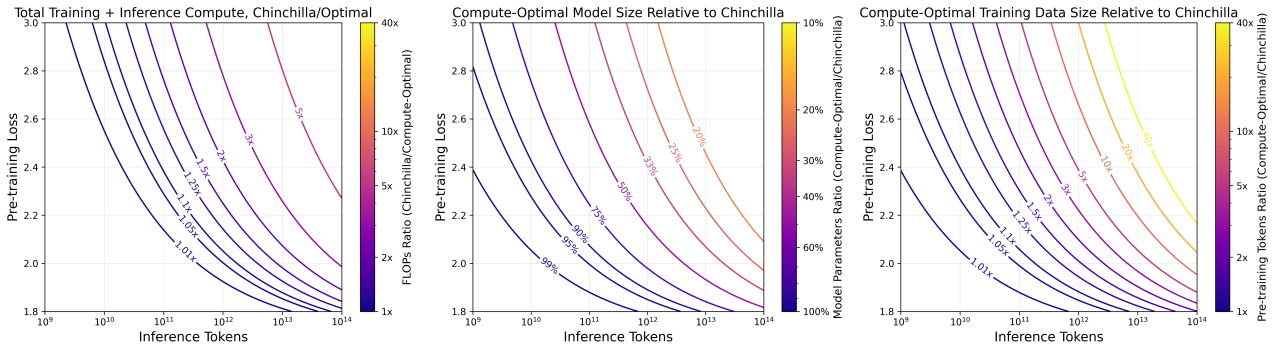
2. Computational Optimality

We seek to minimize the computational costs of a model of a given quality and inference demand. We closely follow the methodology in Hoffmann et al. (2022) (henceforth referred to as "the Chinchilla paper"), using pre-training cross-entropy loss as a proxy for quality, and floating-point operations (FLOPs) as our unit of computational cost.

We model our pre-training loss $L(N, D_{\text{tr}})$ in terms of the number of parameters, N, and pre-training tokens, D_{tr}, according to the Chinchilla paper's third scaling law:

$$L(N, D_{\text{tr}}) \triangleq E + \frac{A}{N^{\alpha}} + \frac{B}{D_{\text{tr}}^{\beta}} \tag{1}$$

The Chinchilla paper derived the parametric loss function in Eq. 1 and fit values for A, B, E, α, and β from the authors' empirical training results. The best-fit values for these coefficients depend on the exact dataset and model architecture; however, the Chinchilla paper found largely consistent results across the MassiveText, Github (Rae et al., 2022), and C4 (Raffel et al., 2023) datasets, and subsequent work has replicated these scaling laws on other internet corpora and transformer variants (Dey et al., 2023; Besiroglu et al., 2024; Gadre et al., 2024). We use the coefficient values from the Chinchilla paper in our analysis here, and



(a) (b) (c)

Figure 2: Ratios of (a) total FLOPs, (b) model parameters, and (c) pre-training tokens, for optimal models estimated via our method vs. Chinchilla-style models. For each point (x, y) in the figures, we compute the Chinchilla model parameter count and training data required to reach the loss y, and the number of combined FLOPs required to train and run inference for x tokens using the Chinchilla model. Then, we compute the same values (total FLOPs, parameter count, training data size) for the compute-optimal models returned by our method, and plot the ratios.

explore fitting these coefficients on different datasets and data ratios more detail in Section 5.

Additionally, we assume that conditioned on pre-training loss, inference demand is independent of model size and token count. In other words, models of equivalent quality but different parameter counts will see the same requests.[1]

Let $\mathrm{T}_{\text{FLOPs}}(N, D)$ and $\mathrm{I}_{\text{FLOPs}}(N, D)$ be the number of FLOPs required to train and run inference, respectively, on a model with N parameters for D tokens. Denote the number of tokens (input + output) of a single inference request i as $D_{\text{inf}}^{(i)}$. Let $D_{\text{inf}} = \sum_i D_{\text{inf}}^{(i)}$ be the sum of all tokens over all inference requests.

Formally, we are interested in minimizing the sum of our training and inference FLOPs under the constraint $L(N, D_{\text{tr}}) = \ell$:

$$N^*(\ell, D_{\text{inf}}), D_{\text{tr}}^*(\ell, D_{\text{inf}}) = \underset{N, D_{\text{tr}} \mid L(N, D_{\text{tr}}) = \ell}{\arg\min} \mathrm{T}_{\text{FLOPs}}(N, D_{\text{tr}}) + \sum_i \mathrm{I}_{\text{FLOPs}}(N, D_{\text{inf}}^{(i)}). \tag{2}$$

N^* and D_{tr}^* are functions that describe the optimal parameters and pre-training tokens, respectively, that minimize total training and inference compute. The pre-training loss constraint ensures that we minimize compute for a given quality.

We use the standard approximation of FLOPs for transformer models with N parameters: $6N$ per training token and $2N$ per inference token (Kaplan et al., 2020). Thus, our objective simplifies to:

$$N^*(\ell, D_{\text{inf}}), D_{\text{tr}}^*(\ell, D_{\text{inf}}) = \underset{N, D_{\text{tr}} \mid L(N, D_{\text{tr}}) = \ell}{\arg\min} 6ND_{\text{tr}} + 2ND_{\text{inf}}. \tag{3}$$

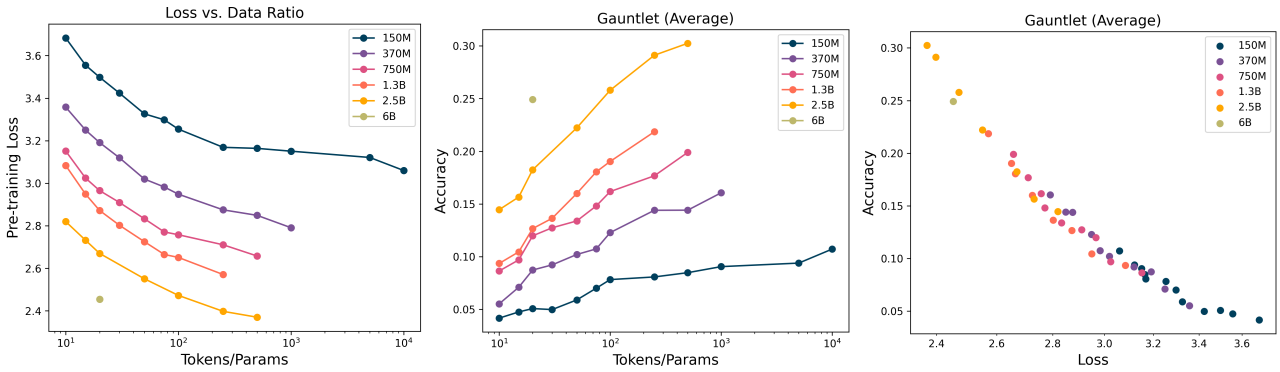
We note that this is the "converse" of the Chinchilla optimization problem. In the Chinchilla paper, the authors assumed a *fixed* compute budget and found N^* and D_{tr}^* that *minimized* pre-training loss. Our objective is to *fix* pre-training loss and find N^* and D_{tr}^* that *minimize* compute costs.

Crucially, our total computational cost depends on the inference demand over the lifetime of the model, but our model's parameter count and data size are determined prior to training. Thus, our analysis is predicated on the assumption that LLM practitioners can estimate their inference demand prior to training.

Without inference ($D_{\text{inf}} = 0$), the optimization problem in Eq. 3 can be solved analytically. Unfortunately, accounting for inference ($D_{\text{inf}} > 0$), determining N^* and D_{tr}^* analytically as functions of ℓ and D_{inf} is intractable (we defer our proof to Appendix A). Instead, we computationally solve for N^* and D_{tr}^* across a range of values of ℓ and D_{inf} using the Newton root-finding method. In practice, this method converges for relevant inputs and we are able to determine optimal parameter/token counts.

In Figure 2, we show how our inference-adjusted model's FLOP counts, parameters, and pre-training tokens compare to Chinchilla-style models across a range of loss values and inference demands. When inference usage is significantly less than the number of pre-training tokens, Chinchilla models are essentially compute-optimal. However, as demand

[1] In practice, smaller models of equivalent quality may have greater demand since they can have lower inference latency.



(a) (b) (c)

Figure 3: For each model (150M, 370M, 750M, 1.3B, 2.5B, 6B) in our experimental sweep, we plot (a) Loss vs. Tokens per parameter, (b) Gauntlet Average (an aggregation of all our metrics described in Sec. 3) vs. Tokens per parameter, and (c) Loss vs. Gauntlet Average. Category-specific Gauntlet results are available in Sec. D.

increases, inference costs becomes a significant factor. For a 7B-Chinchilla-quality model with an inference demand of 10^{11} tokens, our formula suggests the compute-optimal method is to train a 6B parameter model on 1.18× the original (Chinchilla-prescribed) amount of data. For higher quality models (i.e. models that are larger and/or trained for longer), the volume of inference demand required to shift the scaling law increases: An LLM developer that expects a 30B-Chinchilla-quality model will see 10^{13} tokens during inference can reduce their total FLOPs by 28% by training a 13.6B model on 2.84× the data. We provide additional results in Sec. B.1 in the Appendix.

3. Experiments

In high-demand inference scenarios, our analysis in Section 2 suggests that we should train models significantly smaller and on much more data than Chinchilla-style, resulting in hundreds or even thousands of tokens per parameter. The Chinchilla scaling laws have no upper bound—their form infinitely predicts that as tokens per parameter increase, model quality increases. However, the Chinchilla authors do not validate the scaling law at the outer ranges, conducting experiments only at typical (<~100 tokens/parameter) ratios. De Vries (2023) postulates that there is a critical model size (~30%), below which, it is not possible to train on *any* number of tokens and match a Chinchilla-style model.

To characterize the behavior of transformers at extreme data sizes, we train 47 models with the MPT architecture (MosaicML, 2023) of varying size and token ratios. Our models range from 150M to 6B parameters, and our data budgets from 10 to 10,000 tokens per parameter. Due to resource constraints, we could not complete a full sweep for all model sizes (e.g. we train our 2.5B model up to 500 tokens/parameter). Our dataset consists of trillions of tokens of general web text and code. For all experiments, we train for only a single epoch and do not repeat data. Further details are provided in Section C in the Appendix.

Furthermore, to ensure that loss is a good proxy for downstream metrics, we evaluate each model on a version of the open source Evaluation Gauntlet (MosaicML NLP Team, 2023), with tasks in five categories:

- **World Knowledge**: This includes Jeopardy (Wolfe et al., 2022), MMLU (Hendrycks et al., 2020), BIG-bench WikiData (Srivastava et al., 2022), ARC Easy and ARC Challenge (Clark et al., 2018).
- **Commonsense Reasoning**: BIG-bench Strategy QA and BIG-bench Strange Stories (Srivastava et al., 2022), COPA (Roemmele et al., 2011), PIQA (Bisk et al., 2020), OpenBook QA (Mihaylov et al., 2018), Common Sense QA (Talmor et al., 2018), and SIQA (Sap et al., 2019).
- **Reading Comprehension**: SQuAD (Rajpurkar et al., 2016), BoolQ (Clark et al., 2019), CoQA (Reddy et al., 2019), and AGI Eval (Zhong et al., 2023).
- **Language Understanding**: LAMBADA (Paperno et al., 2016), HellaSwag (Zellers et al., 2019), Winograd Schema Challenge (Levesque et al., 2012), and Winogrande (Sakaguchi et al., 2019).
- **Symbolic Problem Solving**: BIG-bench Elementary Math QA (Srivastava et al., 2022) BIG-bench Dyck Languages (Srivastava et al., 2022), BIG-bench Operators (Srivastava et al., 2022), Math QA (Amini et al., 2019), LogiQA, GSM8k (Cobbe et al., 2021), SVAMP (Patel et al., 2021), AGI Eval SAT Math and AGI Eval LSAT (Zhong et al., 2023).

To compute model performance on the above datasets, the Evaluation Gauntlet uses the InContextLearningQAAccuracy for question-answering tasks, InContextLearningLMAccuracy for language-modeling tasks, and InContextLearningMultipleChoiceAccuracy for multiple-choice tasks. All metrics are from the Composer library (The Mosaic ML Team, 2021).

The Gauntlet Average is an average of the accuracy of all the above tasks, with each task weighted equally after subtracting out the task's baseline (random) accuracy and normalizing.

4. Results

In Figure 3, we show the major results of our experiments. Our key finding is that loss continues to decrease as we increase tokens per parameter, even to extreme ratios. Although it takes exponentially more tokens to reduce loss at large ratios, loss does not plateau as we scale to 10,000 tokens per parameter for our 150M model. For models larger than 150M, we only test up to a maximum of 1,000 tokens per parameter due to resource constraints, and at this scale we also see no evidence of loss flat-lining. Further experiments are needed to see how loss scales beyond this point.

Existing literature claims there exists a critical model size, below which we cannot match a Chinchilla-optimal model's quality (De Vries, 2023). Since our experiments do not see any plateauing of loss as we scale token ratios, we find no evidence to support the critical size hypothesis, although further testing is needed at extreme scales—it is possible that beyond 10,000 tokens/parameter, behavior changes.

We also show results for downstream metrics. Fig. 3b shows that the Gauntlet Average improves as we increase tokens per parameter — again, we find no evidence of a "saturation" point, beyond which additional tokens do not result in better performance. In fact, as Fig. 3c shows, as loss decreases, smaller decreases in loss lead to larger improvements in downstream accuracy.

In Fig. 3c, we plot the Gauntlet Average as a function of loss. Loss and Gauntlet Average are tightly correlated, showing that improvements in loss are excellent predictors for improvements in general model quality. Our results show that practitioners interested in predicting downstream metrics as a function of model parameters and token counts can make use of existing scaling laws to accurately understand how their downstream metrics change at scale.

In Figure 4, we plot loss vs. FLOPs, grouping our data points by token-per-parameter ratio (up to 500 tokens/param) instead of model size, following Gadre et al. (2024). For token-per-parameter ratios ≥ 20, our lines of best fit are nearly parallel, indicating that models learn with similar efficiency. This result is striking, as it indicates that loss decreases at similar rates for each additional FLOP regardless of whether training occurs in the standard Chinchilla regime or at extreme ratios (500 tokens/param). Our results also confirm that training at low (< 20) token-per-parameter ratios is less computationally efficient, in line with the results from the Chinchilla paper.

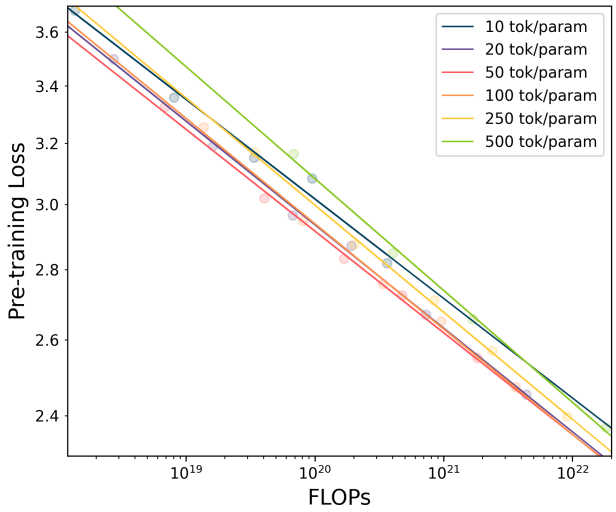


Figure 4: Log-log plot of loss vs. FLOPs, grouped by token/parameter ratio. The slope of each trendline shows how efficiently models learn at each ratio.

In Section D in the Appendix, we show results for each Gauntlet category. Since they consist of fewer tasks, category averages show less consistent correlation with data and model sizes than the overall Gauntlet Average, because on many tasks smaller models are not able to achieve significantly better performance than a random baseline (Barton, 2024).

5. Parametric Fitting

The Chinchilla scaling law has five coefficients (A, B, E, α and β) which were found empirically by fitting Eq. 1 to data collected from 400 training runs by Hoffmann et al. (2022). These coefficients determine precisely how much the scaling law weighs increasing model parameters vs. increasing training data. Muennighoff et al. (2023) found similar coefficient values after training 54 similarly-sized runs on C4, one of the datasets tested in the Chinchilla paper. In both cases, the empirical data collected was largely from small token-to-parameter training runs ($<\sim 100$). Since Hoffmann et al. (2022) and Muennighoff et al. (2023) focused on optimizing training compute and data efficiency, respectively, it is reasonable that they concentrated empirical data col-

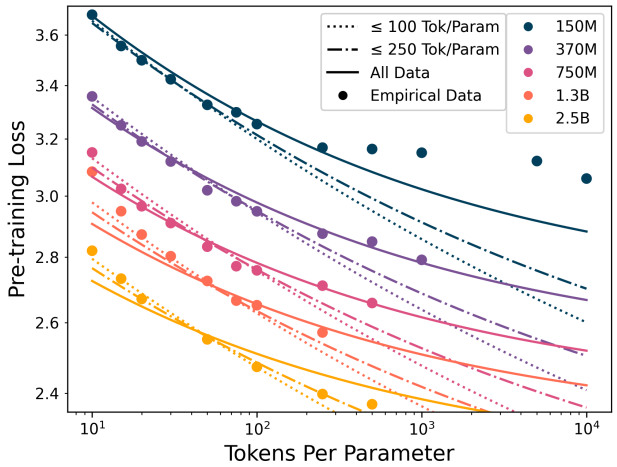


Figure 5: Chinchilla parameteric scaling curves fit to progressively larger subsets of our empirical training data. The coefficients that define these curves are detailed in Table 1.

lection around the training efficiency "sweet spot," around 20 tokens per parameter. However, when we incorporate inference costs into the picture, we may need to train models for hundreds or thousands of tokens per parameter. Thus, we require scaling laws that can accurately predict training loss far beyond typical data ratios.

To understand how well the Chinchilla parametric fitting procedure generalizes to loss values for extremely long training runs, we fit parametric curves on successively larger subsets of our training data. First, we use only training runs with no more than 100 tokens per parameter, followed by runs with no more than 250 tokens per parameter, etc. For each of the curves, we follow the procedure described in the Sec. D.2 of the Chinchilla paper, minimizing the objective in Eq. 4, where LSE is the log-sum-exp operator, and $\delta = 10^{-3}$. We use the L-BFGS algorithm to perform the minimization, initialized from a grid of starting points for each of the parameters. We then set $A, B, E = \exp(a), \exp(b), \exp(e)$ and minimize the Huber loss in the following equation:

$$\min_{a,b,e,\alpha,\beta} \sum_{\text{Run } i} \text{Huber}_\delta \Big(\text{LSE}\big[a - \alpha \log(N^{(i)}), \; b - \beta \log(D_{\text{tr}}^{(i)}), e\big] - \log(L^{(i)}) \Big) \tag{4}$$

Our results are shown in Table 1 and visualized in Figure 5. First, we see that our parameteric curves follow a consistent trend—as we use more extreme data in our fitting procedure, our scaling curves become flatter. This trend suggests that if we only use data from typical token ratios to determine our scaling law coefficients, we will *overestimate* the impact of additional training data as we move towards the long-data-ratio regime. Taken together with our results from Sec. 4, our experiments show that although models continue to learn at extreme training durations, they do so more slowly than scaling laws predict.

Table 1: Parametric fitting on data including extreme training runs. The Chinchilla coefficients from Hoffmann et al. (2022) are included for reference.

Data	α	β	A	B	E
≤100 tok/param	0.08	0.13	7.199	25.97	0.17
≤250 tok/param	0.13	0.16	14.23	39.54	0.98
≤500 tok/param	0.13	0.16	17.07	35.80	0.95
All Data	0.18	0.24	33.66	138.9	1.45
Chinchilla	0.34	0.28	406.4	410.7	1.69

Second, somewhat surprisingly, none of of our parametric curves fit our 150M long-ratio training results well. It appears that as we extend training duration far beyond typical Chinchilla ratios, the parametric loss function is *not flexible enough* to accurately model the behavior of both smaller (≤150M) and larger models. Further research is needed to understand the limits of scaling law extrapolation.

We note that our experimentally-derived coefficients differ from those found in the Chinchilla paper. This is expected, as both the data and the model architectures we used to train all checkpoints are quite different from the original work (Hoffmann et al., 2022). Besiroglu et al. (2024) noted in a replication study that the coefficients reported in Hoffmann et al. (2022) were rounded in ways that leads to significant bias in scaling law predictions.

Since Hoffmann et al. (2022) trained almost exclusively on small-duration training runs (≤~100 tokens/parameter), our results suggest that that in their original form, the Chinchilla scaling laws do not extend to extreme-duration training runs. To the extent that they do, current scaling laws overestimate the improvements in loss that stem from long-duration training on additional data.

These results have significant impact as the field continues to train longer and longer. For example, LLaMA 7B was trained on 1 trillion tokens (Touvron et al., 2023a), Llama 2 7B was trained on 2 trillion tokens (Touvron et al., 2023b), and Llama 3 8B was trained on 15 trillion tokens (AI@Meta, 2024). How much of the quality difference between these models is due to simply training on more data? How much is due to other changes, like architecture modifications or data quality improvements? Our results indicate that if we apply the Chinchilla scaling laws to understand the scaling component of this quality improvement, we will likely overestimate the its impact of more data versus the other changes.

Further work is needed to fully characterize scaling laws

at extreme token to parameter ratios. Due to resource constraints, we do not collect data at the same scale as the Chinchilla paper—both in terms of model size (we only test up to 6B vs. 16B), and number of training runs (47 vs. 400). Little research has been conducted to explore the minimum data required to accurately fit scaling laws coefficients (see Besiroglu et al. (2024)).

6. Estimating Real-World Cost Optimality

Our method introduced in Section 2 optimizes purely for minimum total (training + inference) FLOPs. However, this has significant drawbacks which limit its applicability to real-world deployments. The real-world cost of an inference request of $3D$ tokens is generally different than the cost to train on D tokens. For instance, inference hardware utilization can be much lower than training utilization, since small batch size computation can result in low Model FLOPs Utilization (MFU). MFU can be as low as ~1% for inference (Pope et al., 2022) but is typically 40-60% during training (Korthikanti et al., 2022). Utilization is also different for input tokens vs. output tokens — since input tokens (prompts) are typically processed in a single forward pass, utilization is typically near training levels. By contrast, during generation, output tokens must be produced sequentially, resulting in low utilization due to memory bandwidth constraints. Another complicating factor is that inference operations can sometimes be cheaper than training FLOPs, since models can be quantized before inference time, turning 16- or 32-bit floating-point operations into 4- or 8-bit integer operations which run more efficiently on the same hardware. Quantization can also enable LLMs to fit on GPUs with less VRAM, so training and inference may occur on different hardware altogether (Frantar et al., 2023).

To estimate the real-world cost of inference, we modify Eq. 2 to account for hardware utilization: MFU_{tr}, $\mathrm{MFU}_{\text{inp}}$, and $\mathrm{MFU}_{\text{out}}$ are our training, inference input, and inference output MFUs, respectively. In addition, we add parameters for training and inference cost per FLOP, C_{tr} and C_{inf}. Our new objective is:

$$N^*(\ell, D_{\text{inp}}, D_{\text{out}}), D^*_{\text{tr}}(\ell, D_{\text{inp}}, D_{\text{out}}) = \underset{N, D \mid L(N, D_{\text{tr}}) = \ell}{\arg\min} \left[\frac{C_{\text{tr}}}{\mathrm{MFU}_{\text{tr}}} \mathrm{T}_{\text{FLOPs}}(N, D_{\text{tr}}) + \sum_i \frac{C_{\text{inf}}}{\mathrm{MFU}_{\text{inp}}} \mathrm{I}_{\text{FLOPs}}(N, D^{(i)}_{\text{inp}}) + \sum_i \frac{C_{\text{inf}}}{\mathrm{MFU}_{\text{out}}} \mathrm{I}_{\text{FLOPs}}(N, D^{(i)}_{\text{out}}) \right]. \tag{5}$$

We again use the approximations for FLOPs for transformer models, reducing the above equation to:

$$N^*(\ell, D_{\text{inp}}, D_{\text{out}}), D^*_{\text{tr}}(\ell, D_{\text{inp}}, D_{\text{out}}) = \underset{N, D_{\text{tr}} \mid L(N, D_{\text{tr}}) = \ell}{\arg\min} \frac{6 N D_{\text{tr}} C_{\text{tr}}}{\mathrm{MFU}_{\text{tr}}} + 2 N C_{\text{inf}} \left[\frac{D_{\text{inp}}}{\mathrm{MFU}_{\text{inp}}} + \frac{D_{\text{out}}}{\mathrm{MFU}_{\text{out}}} \right] \tag{6}$$

Eq. 6 is a simplified model of real-world costs: we leave aside latency requirements and assume MFU and cost per FLOP do not depend on model size, configuration, or sequence length. Still, our approximation is flexible enough to account for heterogeneous hardware utilization and costs.

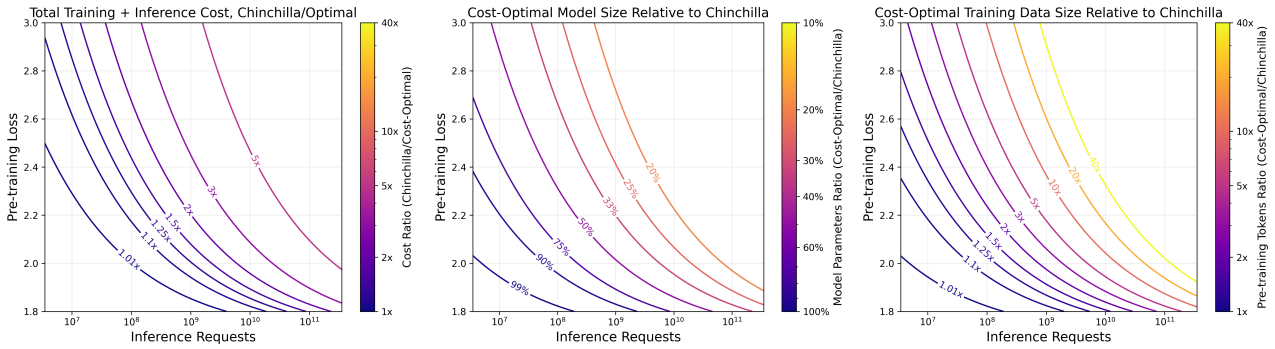
In Figure 6, we show how inference-adjusted cost-optimal models compare to Chinchilla-style models, assuming typical training and inference hardware costs and MFU. For a 30B-Chinchilla-quality model, LLM practitioners expecting 1.5B inference requests can reduce costs by 17% by instead training a 16B model on 3.35T tokens. In Sec. B.2, we show further results for various configurations.

Comparing our compute-optimal analysis in Fig. 2 to our real-world cost analysis in Fig. 6, we see that for the same inference demand of 2T tokens (7.02B requests), a Chinchilla-70B model requires only **1.3%** extra FLOPs compared to an equal-quality *compute-optimal* model, but costs **36%** more than a *cost-optimal* model. This difference is attributable to the 50× lower MFU of each inference output token compared to training, which our FLOP-based analysis in Sec. 2 fails to capture.

7. Related Work

Many studies have contributed to the development of scaling laws for LLMs, including Hestness et al. (2017; 2019), Rosenfeld et al. (2019), Henighan et al. (2020), Kaplan et al. (2020), Sorscher et al. (2022), Tay et al. (2022), and Caballero et al. (2022) (see Villalobos (2023) for a review). Some of these studies focused on scaling laws for transfer settings (i.e. downstream performance), such as Hernandez et al. (2021); Mikami et al. (2021); Abnar et al. (2021) and Tay et al. (2022).

A few studies have also (gently) critiqued the general parametric function fitting approach of Hoffmann et al. (2022). Besiroglu et al. (2024) attempted to replicate the methodology used in Hoffmann et al. (2022) and found that the confidence intervals reported in the original study were implausibly narrow. The broader implication is that confidence intervals are quite wide for parametric function fitting on a small number of data points. This is also a potentially valid critique of our empirical results, as our analysis only includes 47 separate experiments (Chinchilla included more than 400 experiments). In a similar vein, Porian et al. (2024) investigate discrepancies between the scaling laws from Kaplan et al. (2020) and Hoffmann et al. (2022).



(a) (b) (c)

Figure 6: Ratios of (a) total training + inference cost, (b) model parameters, and (c) pre-training tokens, for cost-optimal models via our real-world estimation method vs. Chinchilla-style models. Results in this figure are shown with the following settings: training with 50% MFU, inference input with 50% MFU, generation with 1% MFU. Inference requests have 70 input tokens and 215 output tokens each, aligning with averages from real-world data (Zheng et al., 2023). To mimic a realistic scenario, we calculate costs assuming training occurs on A100-80GB and inference occurs on A100-40GB accelerators after INT8 quantization (see Sec. B.3 for details).

A handful of compelling scaling law papers have been published since 2023, when an earlier version of this work was first presented (Sardana & Frankle, 2023). For example, Krajewski et al. (2024) characterize differences in scaling properties between dense transformers and Mixture of Expert (MoE) models. More theoretical studies include Michaud et al. (2024) and Bordelon et al. (2024). Paquette et al. (2024) uses phase-plane analysis to characterize scaling laws in the compute-limited, infinite-data regime. Ruan et al. (2024) builds scaling laws from multiple LLM model families using low-dimensionality tools applied to publicly available data of LLM performance on open benchmarks.

The results presented in Gadre et al. (2024) are particularly relevant to this paper. The authors train 100 models between the sizes of 1.4B and 6.9B parameters and on data with tokens-per-parameter ratios between 20 and 640. Similar to our study, they find reliable scaling laws in these model and data regimes. They also find that downstream task performance is strongly correlated to LLM perplexity.

8. Conclusion

In this work, we modify the Chinchilla scaling laws to account for both the computational and real-world costs of inference. As inference demand approaches pre-training data size, the additional cost pushes the optimal tokens-to-parameters ratio towards smaller and longer-trained models.

We experimentally validate the hypothesis that very small models, trained on enough data, can match larger Chinchilla-style ones. Both in terms of loss and downstream metrics, model quality improves as tokens per parameter increase. Further work is needed to show if this scales beyond 10,000 tokens per parameter, or at larger model sizes. Still, our results show that practitioners operating in inference-heavy regimes, or with limited deployment memory, can scale training duration considerably longer than current literature suggests and see quality improvements.

Finally, we show evidence that the Chinchilla parametric coefficient fitting procedure overestimates the reduction from additional training data when applied to extremely data-heavy training runs. More work is needed to develop scaling laws that apply precisely at a wide range of ratios.

Acknowledgements

We thank Daya Khudia for his support and Mihir Patel and Linden Li for their feedback on this manuscript.

Impact Statement

This paper presents work whose goal is to reduce the costs of producing large language models. LLMs have the potential to produce factual, high-quality text, but also biased or harmful outputs. By reducing the costs train capable LLMs, we make them more accessible to scientific researchers, industry, and the general population.

References

Abnar, S., Dehghani, M., Neyshabur, B., and Sedghi, H. Exploring the limits of large scale pre-training. *arXiv preprint arXiv:2110.02095*, 2021.

AI@Meta. Llama 3 model card, 2024. URL https://github.com/meta-llama/llama3/

blob/main/MODEL_CARD.md.

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023.

Amini, A., Gabriel, S., Lin, S., Koncel-Kedziorski, R., Choi, Y., and Hajishirzi, H. Mathqa: Towards interpretable math word problem solving with operation-based formalisms. *CoRR*, abs/1905.13319, 2019. URL http://arxiv.org/abs/1905.13319.

Barton, T. Calibrating the mosaic evaluation gauntlet. https://www.databricks.com/blog/calibrating-mosaic-evaluation-gauntlet, April 2024. Blog post.

Besiroglu, T., Erdil, E., Barnett, M., and You, J. Chinchilla scaling: A replication attempt. *arXiv preprint arXiv:2404.10102*, 2024.

Bisk, Y., Zellers, R., Gao, J., Choi, Y., et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 7432–7439, 2020.

Bordelon, B., Atanasov, A., and Pehlevan, C. A dynamical model of neural scaling laws. *arXiv preprint arXiv:2402.01092*, 2024.

Caballero, E., Gupta, K., Rish, I., and Krueger, D. Broken neural scaling laws. *arXiv preprint arXiv:2210.14891*, 2022.

Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Liu, Y., Pham, H., Dong, X., Luong, T., Hsieh, C.-J., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms, 2023.

Clark, C., Lee, K., Ming-Wei Chang, T. K., Collins, M., and Toutanova, K. Boolq: Exploring the surprising difficulty of natural yes/no questions. In *NAACL*, 2019.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*, 2018.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

De Vries, H. Go smol or go home, 2023. URL https://www.harmdevries.com/post/model-size-vs-compute-overhead/.

Dey, N., Gosal, G., Zhiming, Chen, Khachane, H., Marshall, W., Pathria, R., Tom, M., and Hestness, J. Cerebras-gpt: Open compute-optimal language models trained on the cerebras wafer-scale cluster, 2023.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. Gptq: Accurate post-training quantization for generative pre-trained transformers, 2023.

Gadre, S. Y., Smyrnis, G., Shankar, V., Gururangan, S., Wortsman, M., Shao, R., Mercat, J., Fang, A., Li, J., Keh, S., et al. Language models scale reliably with over-training and on downstream tasks. *arXiv preprint arXiv:2403.08540*, 2024.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

Henighan, T., Kaplan, J., Katz, M., Chen, M., Hesse, C., Jackson, J., Jun, H., Brown, T. B., Dhariwal, P., Gray, S., et al. Scaling laws for autoregressive generative modeling. *arXiv preprint arXiv:2010.14701*, 2020.

Hernandez, D., Kaplan, J., Henighan, T., and McCandlish, S. Scaling laws for transfer. *arXiv preprint arXiv:2102.01293*, 2021.

Hestness, J., Narang, S., Ardalani, N., Diamos, G., Jun, H., Kianinejad, H., Patwary, M. M. A., Yang, Y., and Zhou, Y. Deep learning scaling is predictable, empirically. *arXiv preprint arXiv:1712.00409*, 2017.

Hestness, J., Ardalani, N., and Diamos, G. Beyond human-level accuracy: Computational challenges in deep learning. In *Proceedings of the 24th symposium on principles and practice of parallel programming*, pp. 1–14, 2019.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., and Sifre, L. Training compute-optimal large language models, 2022.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models, 2020.

Knight, W. Openai's ceo says the age of giant ai models is already over. *Wired*, 2023. ISSN 1059-1028. URL https://www.wired.com/story/openai-ceo-sam-altman-the-age-of-giant-ai-models-is-already-over/.

Korthikanti, V., Casper, J., Lym, S., McAfee, L., Andersch, M., Shoeybi, M., and Catanzaro, B. Reducing activation recomputation in large transformer models, 2022.

Krajewski, J., Ludziejewski, J., Adamczewski, K., Pióro, M., Krutul, M., Antoniak, S., Ciebiera, K., Król, K., Odrzygóźdź, T., Sankowski, P., et al. Scaling laws for fine-grained mixture of experts. *arXiv preprint arXiv:2402.07871*, 2024.

Labs, L. Gpu cloud - vms for deep learning. https://lambdalabs.com/service/gpu-cloud, 2023. Accessed 2023-10-02.

Levesque, H., Davis, E., and Morgenstern, L. The winograd schema challenge. In *Thirteenth International Conference on the Principles of Knowledge Representation and Reasoning*. Citeseer, 2012.

Michaud, E., Liu, Z., Girit, U., and Tegmark, M. The quantization model of neural scaling. *Advances in Neural Information Processing Systems*, 36, 2024.

Mihaylov, T., Clark, P., Khot, T., and Sabharwal, A. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Conference on Empirical Methods in Natural Language Processing*, 2018. URL https://api.semanticscholar.org/CorpusID:52183757.

Mikami, H., Fukumizu, K., Murai, S., Suzuki, S., Kikuchi, Y., Suzuki, T., Maeda, S.-i., and Hayashi, K. A scaling law for synthetic-to-real transfer: How much is your pre-training effective? *arXiv preprint arXiv:2108.11018*, 2021.

MosaicML. Introducing mpt-7b: A new standard for open-source, commercially usable llms, 2023. URL www.mosaicml.com/blog/mpt-7b. Accessed: 2024-01-28.

MosaicML NLP Team. Llm evaluation scores. https://www.mosaicml.com/llm-evaluation, 2023. Accessed: 2023-09-28.

Muennighoff, N., Rush, A. M., Barak, B., Scao, T. L., Piktus, A., Tazi, N., Pyysalo, S., Wolf, T., and Raffel, C. Scaling data-constrained language models, 2023.

NVIDIA. Nvidia a100 datasheet, 2021. URL https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf.

OpenAI and Pilipiszyn, A. Gpt-3 powers the next generation of apps, Mar 2021. URL https://openai.com/blog/gpt-3-apps.

Paperno, D., Kruszewski, G., Lazaridou, A., Pham, Q. N., Bernardi, R., Pezzelle, S., Baroni, M., Boleda, G., and Fernández, R. The lambada dataset: Word prediction requiring a broad discourse context. *arXiv preprint arXiv:1606.06031*, 2016.

Paquette, E., Paquette, C., Xiao, L., and Pennington, J. 4+3 phases of compute-optimal neural scaling laws. *arXiv preprint arXiv:2405.15074*, 2024.

Patel, A., Bhattamishra, S., and Goyal, N. Are NLP models really able to solve simple math word problems? In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2080–2094, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.168. URL https://aclanthology.org/2021.naacl-main.168.

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Levskaya, A., Heek, J., Xiao, K., Agrawal, S., and Dean, J. Efficiently scaling transformer inference, 2022.

Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *arXiv preprint arXiv:2406.19146*, 2024.

Press, O., Smith, N. A., and Lewis, M. Train short, test long: Attention with linear biases enables input length extrapolation, 2022.

Rae, J. W., Borgeaud, S., Cai, T., Millican, K., Hoffmann, J., Song, F., Aslanides, J., Henderson, S., Ring, R., Young, S., Rutherford, E., Hennigan, T., Menick, J., Cassirer, A., Powell, R., van den Driessche, G., Hendricks, L. A., Rauh, M., Huang, P.-S., Glaese, A., Welbl, J., Dathathri, S., Huang, S., Uesato, J., Mellor, J., Higgins, I., Creswell, A., McAleese, N., Wu, A., Elsen, E., Jayakumar, S., Buchatskaya, E., Budden, D., Sutherland, E., Simonyan, K., Paganini, M., Sifre, L., Martens, L., Li, X. L., Kuncoro, A., Nematzadeh, A., Gribovskaya, E., Donato, D., Lazaridou, A., Mensch, A., Lespiau, J.-B., Tsimpoukelli, M., Grigorev, N., Fritz, D., Sottiaux, T., Pajarskas, M., Pohlen, T., Gong, Z., Toyama, D., de Masson d'Autume, C., Li, Y., Terzi, T., Mikulik, V., Babuschkin, I., Clark, A., de Las Casas, D., Guy, A., Jones, C., Bradbury, J., Johnson, M., Hechtman, B., Weidinger, L., Gabriel, I., Isaac, W., Lockhart, E., Osindero, S., Rimell, L., Dyer, C., Vinyals, O., Ayoub, K., Stanway, J., Bennett, L., Hassabis, D., Kavukcuoglu, K., and Irving, G. Scaling language models: Methods, analysis & insights from training gopher, 2022.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring

the limits of transfer learning with a unified text-to-text transformer, 2023.

Rajpurkar, P., Zhang, J., Lopyrev, K., and Liang, P. SQuAD: 100,000+ Questions for Machine Comprehension of Text. *arXiv e-prints*, art. arXiv:1606.05250, 2016.

Reddy, S., Chen, D., and Manning, C. D. CoQA: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 7:249–266, 2019. doi: 10.1162/tacl_a_00266. URL https://aclanthology.org/Q19-1016.

Roemmele, M., Beja, C. A., and Gordon, A. S. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. *Papers from the 2011 AAAI Spring Symposium*, 2011.

Rosenfeld, J. S., Rosenfeld, A., Belinkov, Y., and Shavit, N. A constructive prediction of the generalization error across scales. *arXiv preprint arXiv:1909.12673*, 2019.

Ruan, Y., Maddison, C. J., and Hashimoto, T. Observational scaling laws and the predictability of language model performance. *arXiv preprint arXiv:2405.10938*, 2024.

Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. WINOGRANDE: an adversarial winograd schema challenge at scale. *CoRR*, abs/1907.10641, 2019. URL http://arxiv.org/abs/1907.10641.

Sap, M., Rashkin, H., Chen, D., LeBras, R., and Choi, Y. Socialiqa: Commonsense reasoning about social interactions. *arXiv preprint arXiv:1904.09728*, 2019.

Sardana, N. and Frankle, J. Beyond chinchilla-optimal: Accounting for inference in language model scaling laws. *arXiv preprint arXiv:2401.00448*, 2023.

Shazeer, N. and Freitas, D. d. Introducing character, Dec 2022. URL https://blog.character.ai/introducing-character/.

Sorscher, B., Geirhos, R., Shekhar, S., Ganguli, S., and Morcos, A. Beyond neural scaling laws: beating power law scaling via data pruning. *Advances in Neural Information Processing Systems*, 35:19523–19536, 2022.

Srivastava, A., Rastogi, A., Rao, A., Shoeb, A. A. M., Abid, A., Fisch, A., Brown, A. R., Santoro, A., Gupta, A., Garriga-Alonso, A., et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*, 2022.

Talmor, A., Herzig, J., Lourie, N., and Berant, J. Commonsenseqa: A question answering challenge targeting commonsense knowledge. *arXiv preprint arXiv:1811.00937*, 2018.

Tay, Y., Dehghani, M., Abnar, S., Chung, H. W., Fedus, W., Rao, J., Narang, S., Tran, V. Q., Yogatama, D., and Metzler, D. Scaling laws vs model architectures: How does inductive bias influence scaling? *arXiv preprint arXiv:2207.10551*, 2022.

The Mosaic ML Team. composer. https://github.com/mosaicml/composer/, 2021.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models, 2023a.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M.-A., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., and Scialom, T. Llama 2: Open foundation and fine-tuned chat models, 2023b.

Tow, J., Bellagente, M., Mahan, D., and Riquelme, C. Stablelm 3b 4e1t, 2023. URL https://huggingface.co/stabilityai/stablelm-3b-4e1t.

Villalobos, P. Scaling laws literature review, 2023. URL https://epochai.org/blog/scaling-laws-literature-review.

Villalobos, P. and Atkinson, D. Trading off compute in training and inference, 2023. URL https://epochai.org/blog/trading-off-compute-in-training-and-inference. Accessed: 2023-9-26.

Wolfe, T., Tunstall, L., and von Platen, P. Jeopardy dataset on hugging face hub. https://huggingface.co/datasets/jeopardy, 2022.

Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. Smoothquant: Accurate and efficient post-training quantization for large language models, 2023.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.

Zheng, L., Chiang, W.-L., Sheng, Y., Li, T., Zhuang, S., Wu, Z., Zhuang, Y., Li, Z., Lin, Z., Xing, E. P., Gonzalez, J. E., Stoica, I., and Zhang, H. Lmsys-chat-1m: A large-scale real-world llm conversation dataset, 2023.

Zhong, W., Cui, R., Guo, Y., Liang, Y., Lu, S., Wang, Y., Saied, A., Chen, W., and Duan, N. Agieval: A human-centric benchmark for evaluating foundation models, 2023.

A. No Analytic Solution for Inference-Compute Optimality

In this section, we prove that it is not possible to analytically derive the optimal model size and pre-training token count according to the third Chinchilla law, after accounting for the computational cost of inference. Conditioned on model quality, we assume that inference demand does not depend on model size and can be estimated prior to training.

Theorem A.1. *Given a fixed model quality and inference demand, there exists no general analytic solution for the compute-optimal model size and pre-training token count according to the third Chinchilla law, after accounting for the computational cost of inference.*

Proof. By Eq. 3, the overall compute cost in FLOPs for training a model with N parameters on D_{tr} tokens and running inference on D_{inf} tokens is given by $C(N, D_{\text{tr}}, D_{\text{inf}}) = 6ND_{\text{tr}} + 2ND_{\text{inf}}$.

We seek the minimum overall compute budget to train and deploy a model of a given quality and inference demand. Formally, we optimize the objective:

$$\min C(N, D_{\text{tr}}, D_{\text{inf}}) \tag{7}$$

subject to the constraint $L(N, D_{\text{tr}}) = E + \frac{A}{N^\alpha} + \frac{B}{D_{\text{tr}}^\beta} = \ell$.

This constraint, from the third Chinchilla law, ensures we are minimizing compute while fixing model quality (pre-training loss). $A = 406.4, B = 410.7, E = 1.69, \alpha = 0.336$, and $\beta = 0.283$ are constants determined empirically by Hoffmann et al. (2022).[2]

We solve this optimization problem via the method of Lagrange multipliers. The gradients are:

$$\nabla C(N, D_{\text{tr}}) = (6D + 2D_{\text{inf}})\hat{i} + 6N\hat{j} \tag{8}$$

$$\nabla L(N, D_{\text{tr}}) = -\alpha A N^{-\alpha-1}\hat{i} - \beta B D_{\text{tr}}^{-\beta-1}\hat{j} \tag{9}$$

We have three equations and three variables ($D_{\text{tr}}, N, \lambda$), where λ is our Lagrange multiplier:

$$6D_{\text{tr}} + 2D_{\text{inf}} = -\lambda\alpha A N^{-\alpha-1} \qquad 6N = -\lambda\beta B D_{\text{tr}}^{-\beta-1} \qquad E + \frac{A}{N^\alpha} + \frac{B}{D_{\text{tr}}^\beta} = \ell \tag{10}$$

With some algebraic manipulation, we can eliminate λ and write $\frac{A}{N^\alpha}$ in terms of D_{tr}:

$$\frac{A}{N^\alpha} = \frac{3\beta B D_{\text{tr}}^{-\beta} + D_{\text{inf}}\beta B D_{\text{tr}}^{-\beta-1}}{3\alpha} \tag{11}$$

We are left to solve the following equation for D_{tr}:

$$0 = (E - \ell) + \left[\frac{\beta B}{\alpha} + B\right] D_{\text{tr}}^{-\beta} + \frac{D_{\text{inf}}\beta B}{3\alpha} D_{\text{tr}}^{-\beta-1} \tag{12}$$

Thus, determining D_{tr} as a function of D_{inf} and ℓ involves finding the roots of equations of the form $ax^{-1.283} + 756.6x^{-0.283} + c = 0$ for arbitrary a and $c > 0$, which is not possible in general.

The best-fit constants A, B, E, α and β vary based on the exact dataset and model architecture. Outside of a handful of special-case values of β for which Eq. 12 can be manipulated into a low-degree polynomial, it is intractable. □

B. Further Results

B.1. Compute-Optimal Results

We present further results from our analysis in Sec. 2. In Table 2, we show the computational cost (in FLOPs) to train and run inference for Chinchilla-style models of various sizes and inference demands. We then calculate the *compute-optimal* model configuration to reach the same quality (equal loss) and run inference, and note the overall compute reduction.

[2]The Chinchilla paper reports $\alpha = 0.34$ and $\beta = 0.28$. However, these are rounded values; to better fit the results reported in Table A.3 of Hoffmann et al. (2022), we use $\alpha = 0.336$ and $\beta = 0.283$, as in De Vries (2023).

Table 2: Compute-Optimal vs. Chinchilla-style Models for Selected Configurations.

		Chinchilla Model			Compute-Optimal Model			
Inference Tokens	Train Loss	Params	Training Tokens	FLOPs	Params	Training Tokens	FLOPs	FLOP Reduction
50B	2.53	1B	27.4B	2.64e20	6.33M	46.8B	2.41e20	9.1%
200B	2.13	7B	276B	1.44e22	5.4B	367B	1.40e22	2.6%
1T	2.05	13B	577B	7.10e22	8.32B	967B	6.49e22	8.5%
5T	1.96	30B	1.56T	5.80e23	16.4B	3.27T	4.86e23	16%
10T	1.89	70B	4.26T	3.19e24	41.6B	7.92T	2.81e24	12%

Table 3: Cost-Optimal vs. Chinchilla-style Models for Selected Configurations.

		Chinchilla Model			Cost-Optimal Model			
Inference Requests	Train Loss	Params	Training Tokens	Total Cost ($)	Params	Training Tokens	Total Cost ($)	Cost Savings
175M	2.53	1B	27.4B	3.77K	327M	152B	1.89K	50%
702M	2.13	7B	276B	124K	2.90B	929B	81.8K	34%
3.51B	2.05	13B	577B	987K	430B	3.1T	500K	49%
17.5B	1.96	30B	1.56T	10.8M	8.58B	12.1T	4.52M	58%
35.1B	1.89	70B	4.26T	51.5M	21.5B	27T	23.8M	54%

B.2. Cost-Optimal Results

We show additional results from our cost-optimality analysis in Sec. 6. In Table 3, we show the total training plus inference costs for Chinchilla models of various sizes at different levels of inference demands. We then calculate costs for equivalent-quality (i.e. same pre-training loss) *cost-optimal* models and show the overall savings. We use the same settings from Figure 6, designed to mimic a typical real-world deployment: training and inference input at 50% MFU, generation at 1% MFU (Korthikanti et al., 2022; Pope et al., 2022). Each inference request has 70 input tokens and 215 output tokens, in accordance with averages from the LMSYS-Chat dataset of 1M inference requests from Zheng et al. (2023). Costs are calculated assuming training and inference on A100-80GB and A100-40GB accelerators, respectively. We further assume the model parameters are quantized to eight-bit integers prior to inference, which is commonly done with no quality reduction (Xiao et al., 2023). All costs are reported in US dollars.

B.3. GPU Details

GPU pricing varies based on vendor and fluctuates over time. At the time of writing, an A100-40GB costs USD $1.10/hr and an A100-80GB costs $1.50/hr on Lambda Labs (Labs, 2023). We use these values in our cost analysis in Sec. 6 and in Table 3. Both variants have a peak performance of 3.12×10^{14} dense FP16/BF16 operations and 6.24×10^{14} INT8 operations per second (NVIDIA, 2021).

C. Model Training

We train MPT-style transformer models (MosaicML, 2023) ranging in size from 150M to 6B parameters, on token/parameter ratios from 10 to 10,000. In Table 4, we provide training configuration details. All models were trained with ALiBi (Press et al., 2022), Grouped Query Attention (Ainslie et al., 2023), the Lion optimizer (Chen et al., 2023) ($\beta_1 = 0.9, \beta_2 = 0.95$) with weight decay equal to the learning rate, cosine warmup ($\alpha_f = 0.1$) with a duration equal to 3 times the number of model parameters, and norm gradient clipping (threshold = 1). A maximum sequence length of 4096 tokens was used. A smaller batch size was used for smaller models so that low-token-count training runs see enough update steps to learn properly. For all experimental results in this work, we use the smoothed final training loss over the last ten batches to reduce

Table 4: Model Training Configurations.

Name	Params	d_model	n_heads	n_layers	Learning Rate	Batch Size	Tokens/Parameter
150M	151M	768	12	12	4.603e-4	160[3]	10,15,20,30,50,75,100, 250,500,1000,5000,10000
370M	367M	1024	16	24	3.453e-4	320	10,15,20,30,50,75,100, 250,500,1000
750M	749M	1536	12	24	2.302e-4	480	10,15,20,30,50,75,100,250,500
1.3B	1.26B	2048	16	24	1.726e-4	960	10,15,20,30,50,75,100,250
2.5B	2.46B	2560	20	32	1.381e-4	960	10,15,20,50,100,250,500
6B	6.05B	4096	32	32	8.632e-5	960	20

noise from minor batch-level variations.

In the rightmost column of Table 4, we list our experiment training token ratios. Note that for the larger models, our training durations are limited by our computational resources.

D. Experiment Results

We present more results from our experiments described in Section 3. In Figure 7, we show aggregate results for each Gauntlet category. Within each category, every task (the tasks are enumerated in Sec. 3) is weighted equally and averaged after subtracting out baseline and normalizing so the maximum achievable accuracy is 1. Note that for some categories (e.g. Symbolic Problem Solving), all models achieve nearly zero performance on all tasks, resulting in low correlation between accuracy and token/parameter ratios (see Barton (2024)).

[3]The 10,000 token/parameter 150M model was trained with batch size 960 due to compute issues.

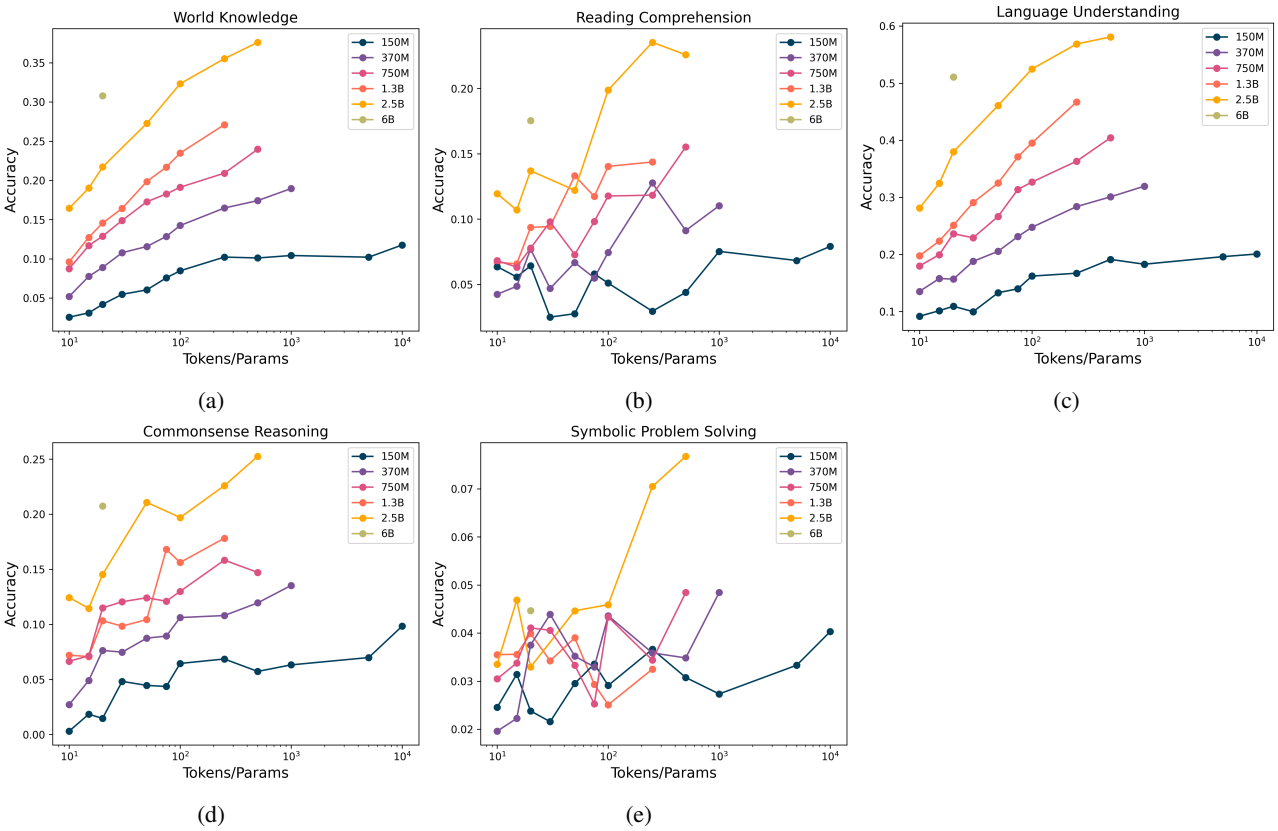


Figure 7: Per-category Gauntlet results.